ĐẠI HỌC QUỐC GIA TP HCM

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÁO CÁO GIỮA KỲ

Môn: **Hệ CSDL tiên tiến**

Thực hiện **Nhóm 1**:

PHẠM THỊ THÚY NGỌC – 250104046

PHẠM THÚY VI – 250104058

CUNG PHƯƠNG ANH – 250104029

PHẠM THỊ THU VÂN - 250104057

Giảng viên**: PGS.TS NGUYỄN ĐÌNH THUÂN**

TP HỒ CHÍ MINH – NĂM 2026

Bảng phân công công việc

Họ và tên	MSSV	Bảng phân công
Phạm Thị Thuý Ngọc	250104046	Nội dung chính (Đặt vấn đề & 4 Lỗ hổng kỹ thuật)
Phạm Thuý Vi	250104058	Code & Tái lập Thực nghiệm
Cung Phương Anh	250104029	Hướng phát triển từ bài báo
Phạm Thị Thu Vân	250104057	Phân tích tính SOTA của bài báo

BÁO CÁO GIỮA KỲ.....1

1. Nội dung chính 3
1.1. Vấn đề bài báo giải quyết 3
1.2. Bốn lỗ hổng trong pipeline học máy của LQO 4
1.3. Kết quả và điều nhóm rút ra 5
2. Tính SOTA của bài báo 5
3. Phát triển từ bài báo (What, How, Why) 6
3.1. WHAT – Bài toán nhóm đề xuất 6
3.2. HOW – Cách thực hiện 7
3.3. WHY – Tại sao làm như vậy 8
4. CÀI ĐẶT MINH HỌA VÀ KẾ HOẠCH THỰC NGHIỆM 8
4.1. Môi trường 8
4.2. Mã nguồn đã chuẩn bị 8
4.3. Thiết kế thực nghiệm và giả thuyết 9
4.4. Chi tiết bài toán phân loại 9
4.5. Kế hoạch chạy và hạn chế 10

1. Nội dung chính

1.1. Vấn đề bài báo giải quyết

Với một câu SQL, bộ tối ưu truy vấn phải chọn một kế hoạch thực thi (thứ tự join, thuật toán join, cách scan) trong một không gian có thể lên tới O(n!) kế hoạch. PostgreSQL tìm kiếm bằng quy hoạch động dựa trên cost model, và chuyển sang thuật toán di truyền GEQO khi truy vấn có từ 12 bảng trở lên. Cost model lại dựa vào ước lượng cardinality vốn thường sai, nên kế hoạch được chọn có thể chậm.

Bộ tối ưu truy vấn học máy (Learned Query Optimizer – LQO) thay cost model bằng mô hình học từ thời gian chạy thật. Có hai nhóm chính: sinh trực tiếp kế hoạch (Neo [2], Balsa [4], LEON [5], HybridQO [6]) và chọn bộ gợi ý (hint set) để PostgreSQL tự sinh kế hoạch (Bao [3]). Các bài báo gốc đều báo cáo LQO nhanh hơn PostgreSQL.

Lehmann và cộng sự [1] đặt câu hỏi: **khi được đánh giá công bằng và tính đủ mọi chi phí, LQO có thật sự tốt hơn PostgreSQL không?** Theo cách hiểu của nhóm, điểm mới của bài là coi LQO như một pipeline học máy thông thường gồm bốn giai đoạn (sinh dữ liệu → mã hóa → huấn luyện → đánh giá), rồi kiểm tra từng giai đoạn có vi phạm nguyên tắc học máy cơ bản hay không. Trong báo cáo này, ký hiệu [1, §x] chỉ mục x của bài báo.

1.2. Bốn lỗ hổng trong pipeline học máy của LQO

Giai đoạn	Lỗ hổng (theo cách hiểu của nhóm)	Bằng chứng trong bài báo	Khuyến nghị
(1) Sinh dữ liệu [1, §3]	Thu hẹp không gian kế hoạch (tắt nested loop, bitmap scan, cây bushy) cho dễ học. Trộn số đo cold/hot cache nên cùng một đầu vào có nhiều nhãn.	Tắt bitmap/tid scan: 28a nhanh hơn 5,5× nhưng 28b chậm hơn 1,9× [1, §8.4]. Lần chạy 2 nhanh hơn lần 1 trung bình 14,6% [1, §8.6].	Giữ nguyên không gian kế hoạch. Chạy mỗi câu 3 lần, lấy lần 3 (hot cache).
(2) Mã hóa [1, §4]	Mã hóa filter chỉ bằng cardinality nên hai truy vấn khác nhau có thể cùng vector đặc trưng nhưng khác thời gian chạy. Bao chỉ mã hóa kế hoạch, không mã hóa truy vấn.	Vi phạm nguyên tắc bất biến: cùng đầu vào phải cho cùng đầu ra, tức ánh xạ đặc trưng → nhãn phải là một hàm [1, §4.1].	Mã hóa cả truy vấn lẫn kế hoạch; vector hóa điều kiện lọc tường minh.
(3) Huấn luyện [1, §5]	Không có tập validation cố định. Pre-train bằng cost rồi chuyển sang latency	LEON train 110–130 giờ, Balsa 40–85 giờ nhưng	Dùng tập holdout cố định; dùng learning-

	giữa chừng làm mất kiến thức, vì hai đại lượng khác thang đo.	xếp cuối; Bao chỉ train khoảng 2 giờ [1, §8.2.2].	to-rank thay cho việc đổi mục tiêu.
(4) Đánh giá [1, §6, §7]	Các biến thể của cùng một truy vấn gốc (1a, 1b, …) nằm ở cả train và test gây rò rỉ dữ liệu. Không tính thời gian suy luận.	Chia theo số join không giải thích được thời gian chạy ($R^2 = -0{,}11$) [1, §6.1]. LEON thực thi tập test 58 s nhưng suy luận mất khoảng 9,6 giờ [1, §8.2.1].	Ba kiểu chia: Leave-One-Out (dễ), Random (trung bình), Base Query (khó). Đo end-to-end.

1.3. Kết quả và điều nhóm rút ra

Khi cùng phần cứng, cấu hình và cách chia dữ liệu, PostgreSQL xếp thứ nhất trên cả JOB và STACK, theo thứ tự PostgreSQL > HybridQO > Bao > Neo > Balsa > LEON [1, §8.2.1]. Bài báo kết luận các LQO hiện tại *không tốt hơn PostgreSQL một cách nhất quán*. Nhóm rút ra ba điều:

- Phần lớn cải thiện được báo cáo trong các bài LQO gốc đến từ cách thiết lập thí nghiệm (data-centric) hơn là từ mô hình.
- Thước đo đúng là tổng thời gian người dùng phải chờ (suy luận + lập kế hoạch + thực thi), không chỉ thời gian thực thi.
- Muốn biết LQO có tổng quát hóa được hay không thì phải kiểm thử trên các truy vấn gốc chưa từng thấy khi huấn luyện (Base Query split).

2. Tính SOTA của bài báo

Bài báo không đề xuất phương pháp mới nên không đạt SOTA về hiệu năng. Điểm nổi bật nằm ở cách đánh giá. Nhóm tác giả đánh giá lại các bộ tối ưu truy vấn dựa trên học máy hàng đầu như Neo, Bao, Balsa, LEON và HybridQO [1]–[5] trong cùng một điều kiện. Trước đó, mỗi nghiên cứu thường sử dụng cấu hình hệ quản trị cơ sở dữ liệu, cách chia dữ liệu và cách đo thời gian khác nhau, khiến kết quả khó so sánh trực tiếp.

Để giải quyết vấn đề này, nhóm tác giả xây dựng một khung đánh giá thống nhất về cấu hình hệ quản trị cơ sở dữ liệu, dữ liệu, mã nguồn, cũng như cách chia tập huấn luyện và kiểm thử. Bài báo đưa ra ba cách chia dữ liệu với độ khó tăng dần: giữ lại một biến thể, chia ngẫu nhiên và chia theo truy vấn gốc. Cách chia theo truy vấn gốc giúp tránh rò rỉ thông tin giữa các biến thể của cùng một truy vấn. Thời gian chạy được đo đầy đủ, gồm thời gian suy luận, lập kế hoạch và thực thi truy vấn; thời gian huấn luyện được đo riêng. Ngoài ra, bài báo còn phân tích hai vấn đề trong thiết kế các bộ tối ưu. Thứ nhất, cách mã hóa truy vấn chỉ dựa trên số lượng bản ghi có thể khiến những truy vấn khác nhau có cùng biểu diễn. Thứ hai, việc chuyển mục tiêu huấn luyện từ chi phí ước lượng sang thời gian thực thi giữa chừng có thể khiến mô hình mất đi những gì đã học trước đó.

Kết quả cho thấy PostgreSQL vẫn đạt hiệu năng tốt nhất trên cả hai bộ dữ liệu JOB và STACK. HybridQO và Bao có kết quả tương đương, đôi khi nhỉnh hơn PostgreSQL ở một số cách chia, nhưng không vượt trội một cách ổn định. Các phương pháp còn lại có kết quả kém hơn đáng kể. Chẳng hạn, riêng bước suy luận của LEON trên tập kiểm thử của JOB đã mất khoảng 9,6 giờ. Đáng chú ý, các phương pháp có thời gian huấn luyện dài hơn lại cho kết quả kém hơn.

So với nghiên cứu cùng hướng của Zhang và cộng sự [6], vốn tập trung đề xuất một giải pháp thay thế đơn giản hơn, bài báo này tập trung vào việc chuẩn hóa quy trình đánh giá và công khai mã nguồn để người khác có thể tái lập kết quả.

Tuy nhiên, bài báo vẫn có một số hạn chế. Thực nghiệm chỉ sử dụng PostgreSQL 12.5, phạm vi truy vấn còn hẹp và mỗi cách chia dữ liệu chỉ có ba bộ chia. Một số phương pháp như Lero, LOGER và RTOS cũng chưa được đưa vào đánh giá.

Tóm lại, bài báo không đề xuất phương pháp mới đạt SOTA về hiệu năng. Đóng góp chính nằm ở việc xây dựng một khung đánh giá thống nhất, giúp so sánh các bộ tối ưu truy vấn dựa trên học máy công bằng và đáng tin cậy hơn.

Từ những hạn chế được chỉ ra trong [1], báo cáo đề xuất định hướng LQO dựa trên biểu diễn đồng thời query và plan, học xếp hạng các kế hoạch ứng viên, và lấy tổng thời gian xử lý đầu cuối làm tiêu chí lựa chọn và đánh giá. Base Query Sampling, quy trình đo hot cache và cấu hình PostgreSQL nhất quán được sử dụng như các biện pháp kiểm chứng khả năng tổng quát hóa và tính công bằng của kết quả.

Giá trị kỳ vọng của hướng nghiên cứu là một LQO có khả năng mang lại cải thiện ổn định, chi phí suy luận chấp nhận được và kết quả có thể tái lập. Tuy nhiên, đây mới là đề xuất kỹ thuật. Để kết luận mô hình có ưu thế so với PostgreSQL, cần triển khai nguyên mẫu và tiến hành thí nghiệm trên JOB, STACK với các tập chia và điều kiện đo được công bố rõ ràng.

3. Phát triển từ bài báo (What, How, Why)

3.1. WHAT – Bài toán nhóm đề xuất

Bài báo cho thấy không LQO nào tốt hơn PostgreSQL một cách ổn định: ngay cả HybridQO và Bao cũng chỉ nhanh hơn ở một số câu và chậm hơn ở các câu khác [1, §8.2.1]. Vì vậy, thay vì xây một LQO phức tạp thay thế hoàn toàn bộ tối ưu, nhóm đề xuất một bài toán nhỏ hơn, chạy được trên máy cá nhân:

Với mỗi câu SQL, dự đoán trước khi chạy: nên để PostgreSQL tự chọn kế hoạch, hay nên "gợi ý" PostgreSQL tắt một loại toán tử để có kế hoạch nhanh hơn.

Thành phần	Mô tả

Loại bài toán	Phân loại có giám sát, 6 lớp
Input	Một câu SQL, biểu diễn bằng các con số đếm được từ câu lệnh (số bảng, số điều kiện join, số điều kiện lọc, …) và từ kế hoạch mặc định mà PostgreSQL dự kiến chạy (lệnh EXPLAIN: số Hash/Merge/Nested Loop join, loại scan, cost, số dòng ước lượng, …). Không cần chạy câu truy vấn.
Output	Một trong 6 hint set: **default** (giữ nguyên PostgreSQL), **no_nestloop**, **no_hashjoin**, **no_mergejoin**, **no_seqscan**, **no_indexscan**. Mỗi hint set tắt một loại toán tử, ví dụ no_nestloop là lệnh SET enable_nestloop = off.
Nhãn đúng (khi huấn luyện)	Chạy thử câu đó với cả 6 hint set; hint set có tổng thời gian nhỏ nhất là nhãn đúng.
Mô hình	Decision Tree và Random Forest
Mục tiêu	Tổng thời gian chạy cả workload nhỏ hơn so với luôn dùng PostgreSQL, và ít câu bị chậm đi.

Luồng xử lý khi dùng mô hình:

Câu SQL → EXPLAIN (kế hoạch mặc định) → trích đặc trưng (**input**) → **mô hình phân loại** → hint set (**output**), vd no_nestloop → PostgreSQL chạy với hint set đó

Ví dụ minh họa: một câu join 5 bảng mà PostgreSQL ước lượng số dòng quá nhỏ nên dự kiến dùng nhiều Nested Loop. Nếu mô hình đã học được rằng các câu có đặc điểm này thường chạy nhanh hơn khi tắt nested loop, nó trả về no_nestloop; PostgreSQL khi đó chuyển sang Hash Join. Với câu đơn giản, mô hình trả về default để không làm chậm câu. Hiệu quả của mô hình là giả thuyết cần kiểm chứng; hướng mở rộng sau này là xếp hạng các kế hoạch ứng viên bằng Learning-to-Rank như khuyến nghị của [1, §5.2].

3.2. HOW – Cách thực hiện

Bước	**Cách làm**	**Dựa trên [1]**
1. Tạo dữ liệu	Chạy mỗi câu với 6 hint set, mỗi hint set 3 lần, lấy lần 3 (hot cache), timeout 3 phút. Hint set nhanh nhất là nhãn.	Đo ở hot cache [1, §7.3, §8.6]
2. Trích đặc trưng	Lấy từ câu SQL và từ kế hoạch EXPLAIN mặc định, tức mã hóa cả truy vấn lẫn kế hoạch (chi tiết ở Mục 4.4).	Mã hóa cả truy vấn và kế hoạch [1, §4.2]

3. Huấn luyện	Decision Tree và Random Forest; chia dữ liệu theo Leave-One-Out, Random và Base Query.	Ba kiểu chia, tránh rò rỉ [1, §7.2]
4. Đánh giá	Tổng thời gian workload khi chạy theo hint set mô hình chọn, so với "luôn dùng PostgreSQL" và "oracle"; không dùng accuracy làm chỉ số chính.	Accuracy không phù hợp [1, §5.1]
Dữ liệu	JOB 113 câu và khoảng 200 câu CEB-IMDb trên cùng CSDL IMDB (timeout 60 s để vừa thời gian máy).	Benchmark JOB [1, §3.1]

3.3. WHY – Tại sao làm như vậy

- **Tối ưu phải xét tổng chi phí.** Người dùng chờ cả thời gian suy luận lẫn thời gian thực thi. LEON tìm được kế hoạch tốt nhưng suy luận mất 9,6 giờ [1, §8.2.1]. Mô hình phân loại nhỏ chỉ mất vài mili giây để dự đoán.
- **LQO chỉ có lợi ở một phần các câu.** HybridQO và Bao đôi khi nhanh hơn PostgreSQL nhưng không ổn định [1, §8.2.1]. Nếu dự đoán được câu nào nên đổi kế hoạch thì giữ được phần lợi và tránh phần thiệt. Vì "default" luôn là một lựa chọn, mô hình có thể học cách giữ nguyên PostgreSQL khi không chắc chắn.
- **So sánh phải công bằng.** Cache, cấu hình và cách chia dữ liệu có thể đảo ngược thứ hạng các phương pháp [1, §8]. Làm theo đúng quy trình của [1] giúp tách đóng góp thật của mô hình khỏi tác động của thiết lập thí nghiệm.

4. CÀI ĐẶT MINH HỌA VÀ KẾ HOẠCH THỰC NGHIỆM

Nhóm dùng thêm mã nguồn và log mà tác giả công bố tại github.com/edualc/lqo_ml_perspective.

4.1. Môi trường

Thành phần	**Bài báo [1, §8.1]**	**Nhóm**
Phần cứng	x86, 16 CPU, 64 GB RAM, GPU T4	MacBook M1 Pro (ARM64), 8 CPU, 16 GB RAM, không GPU
DBMS	PostgreSQL 12.5 + pg_hint_plan	Giống bài báo, build từ mã nguồn cho ARM64 (Docker)
shared_buffers / work_mem / effective_cache_size	32 GB / 4 GB / 32 GB	3 GB / 128 MB / 8 GB

GEQO, bitmap/tid scan, autovacuum	GEQO và scan bật; tắt autovacuum, ANALYZE một lần	Giống bài báo
Dữ liệu	IMDB, JOB 113 câu	IMDB 21 bảng (12 GB), JOB 113 câu

4.2. Mã nguồn đã chuẩn bị

Thành phần	**Chức năng**
infra/	Dockerfile PostgreSQL 12.5 + pg_hint_plan + pg_bao cho ARM64, Bao server, cấu hình cho 16 GB RAM
run_baseline.sh, run_bao_split.sh, run_and_log.py	Chạy PostgreSQL, Bao và 6 hint set theo quy trình của tác giả (3 lần, lấy lần 3, timeout 3 phút); ghi thời gian và cây kế hoạch
extract_features.py, label.py	Trích đặc trưng (input) và gán nhãn hint set tốt nhất (output)
c1_win_loss.py, c2_classify.py	So sánh thắng/hòa/thua với PostgreSQL; huấn luyện và đánh giá mô hình phân loại

4.3. Thiết kế thực nghiệm và giả thuyết

- **Thí nghiệm 1 – Tái lập:** chạy PostgreSQL và Bao trên JOB, đo end-to-end ở hot cache, so từng câu với PostgreSQL. *Giả thuyết H1:* thứ hạng giữ như bài báo, PostgreSQL không thua Bao trên máy nhóm.
- **Thí nghiệm 2 – Ảnh hưởng của cách chia:** so kết quả Bao trên LOO, Random và Base Query. *Giả thuyết H2:* Base Query split cho kết quả kém nhất, vì không còn rò rỉ giữa các biến thể.
- **Thí nghiệm 3 – Chọn kế hoạch (Mục 4.4):** *Giả thuyết H3:* mô hình chọn hint set giảm được tổng thời gian workload so với "luôn dùng PostgreSQL", đồng thời ít câu bị chậm đi.

Kết quả của các LQO còn lại (HybridQO, Neo, Balsa, LEON) sẽ được tính lại từ log tác giả công bố bằng cùng thước đo, để đặt các thí nghiệm trên trong bối cảnh chung.

4.4. Chi tiết bài toán phân loại

Mục này chi tiết hóa bài toán ở Mục 3.1 (cách chia dữ liệu và đánh giá đã nêu ở bảng 3.2). Input gồm các đặc trưng sau, đều có sẵn trước khi chạy câu truy vấn:

Nguồn	**Đặc trưng (input)**
Câu SQL	Số bảng; số điều kiện join; số điều kiện lọc; số LIKE, NOT LIKE, IN, BETWEEN, OR, IS NULL, so sánh khoảng; độ dài câu lệnh

Kế hoạch mặc định (EXPLAIN)	Số Hash Join, Merge Join, Nested Loop; số Seq Scan, Index Scan, Bitmap Scan; số node và độ sâu cây; cây bushy hay left-deep; cost và số dòng ước lượng
Không dùng	Thời gian chạy, số dòng thực tế, sai số ước lượng (q-error), vì các giá trị này chỉ có sau khi chạy câu truy vấn

- **Nhãn:** một hint set chỉ được tính là tốt hơn default nếu nhanh hơn ít nhất 5%, để không gán nhãn theo nhiễu đo. Ngưỡng này khác ngưỡng 10% dùng để tính thắng/thua vì hai ngưỡng phục vụ hai mục đích khác nhau.
- **Mô hình:** Decision Tree (mô hình chính) và Random Forest, so với hai mốc "luôn dùng PostgreSQL" và "oracle" (luôn chọn đúng hint set tốt nhất, là giới hạn trên). Hai mô hình và hai mốc này **không có trong bài báo [1]**, mà là lựa chọn của nhóm: các LQO trong [1] dùng mạng nơ-ron trên cây kế hoạch (Tree-CNN, Tree-LSTM) cần GPU và nhiều giờ huấn luyện, còn bài toán chọn 1 trong 6 hint set với vài trăm câu phù hợp hơn với mô hình phân loại đơn giản, huấn luyện trên CPU trong vài phút.
- **Giải thích kết quả:** Decision Tree cho ra luật đọc được, ví dụ minh họa: "nếu số bảng ≥ 6 và kế hoạch mặc định có từ 3 Nested Loop trở lên thì dùng no_nestloop". Random Forest cho biết mức độ quan trọng của từng đặc trưng (feature importance), ví dụ số Nested Loop trong kế hoạch mặc định có phải là yếu tố quyết định hay không.

4.5. Kế hoạch chạy và hạn chế

Nhóm chỉ đưa vào thí nghiệm những phần chạy được dưới 20 giờ trên MacBook M1 Pro 16 GB, không GPU. Thời gian dưới đây là ước tính, dựa trên kích thước workload và thời gian huấn luyện trong [1, §8.2.2]:

Thí nghiệm	**Ước tính trên máy nhóm**	**Thực hiện**
PostgreSQL baseline, 113 câu × 3 lần	Dưới 1 giờ	Có
6 hint set × 113 câu × 3 lần (khoảng 2.000 lượt chạy)	~4–12 giờ, tùy số câu bị timeout	Có
Phân loại: Decision Tree, Random Forest × 3 kiểu chia	Vài phút	Có
Bao (huấn luyện bằng CPU) trên 3 split LOO-1, RND-1, BQ-1	~2–6 giờ mỗi split (bài báo: ~2 giờ)	Có
CEB-IMDb ~200 câu × 6 hint set × 3 lần, timeout 60 s	~5–11 giờ	Có
HybridQO / Neo / Balsa / LEON	Bài báo: ~20 / 20–40 / 40–85 / 110–130 giờ mỗi split, trên máy có GPU	Không, dùng log tác giả

- **Hạn chế:** bốn LQO còn lại vượt quá 20 giờ mỗi split hoặc cần GPU nên chỉ phân tích lại log của tác giả, không huấn luyện lại. Kế hoạch "tốt nhất" chỉ là tốt nhất trong 6 hint set, chưa phải tối ưu toàn cục. Phần cứng yếu hơn bài báo nên số tuyệt đối sẽ khác.

Tài liệu tham khảo

[1] R. Marcus et al., "Neo: A Learned Query Optimizer," *PVLDB*, vol. 12, no. 11, 2019.
[2] R. Marcus et al., "Bao: Making Learned Query Optimization Practical," in *Proc. SIGMOD*, 2021, pp. 1275–1288.
[3] Z. Yang et al., "Balsa: Learning a Query Optimizer Without Expert Demonstrations," in *Proc. SIGMOD*, 2022, pp. 931–944.
[4] X. Chen et al., "LEON: A New Framework for ML-Aided Query Optimization," *PVLDB*, vol. 16, no. 9, pp. 2261–2273, 2023.
[5] X. Yu et al., "Cost-based or Learning-based? A Hybrid Query Optimizer for Query Plan Selection," *PVLDB*, vol. 15, no. 13, pp. 3924–3936, 2022.
[6] Y. Zhang et al., "Simple Adaptive Query Processing vs. Learned Query Optimizers: Observations and Analysis," *PVLDB*, vol. 16, no. 9, pp. 2962–2975, 2023.